0981. Time Based Key-Value Store

| Category: Binary Search | LeetCode: https://leetcode.com/problems/time-based

Problem

- Set(key, value, timestamp) stores a version; get(key, timestamp) returns the value with the largest timestamp <= the given one, or "".
- Timestamps for set are strictly increasing.

Intuition

- Because timestamps arrive in increasing order, each key's history list is already sorted.
- Get is "rightmost timestamp <= t", a classic upper-bound binary search.
- (A TreeMap.floorEntry also works and is worth mentioning.)

Brute Force

- Scan the key's history backwards: O(n) per get.

Optimal Approach

- Map<String, List<(ts, value)>>.
- Set appends.
- Get: binary search for the last index with ts <= t.

Worked Example

```
set(foo, bar, 1)      foo: [(1,bar)]
get(foo, 1) -> bar
get(foo, 3) -> bar    (last ts <= 3 is 1)
set(foo, bar2, 4)     foo: [(1,bar),(4,bar2)]
get(foo, 4) -> bar2
get(foo, 5) -> bar2
get(foo, 0) -> ""
```

Java Solution

```java
public class TimeMap {
    private record Entry(int ts, String value) {}
    private final Map<String, List<Entry>> store = new HashMap<>();

    public void set(String key, String value, int timestamp) {
        store.computeIfAbsent(key, k -> new ArrayList<>()).add(new Entry(timestamp, value));
    }

    public String get(String key, int timestamp) {
        List<Entry> list = store.get(key);
        if (list == null) return "";
        int lo = 0, hi = list.size() - 1, ans = -1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (list.get(mid).ts() <= timestamp) { ans = mid; lo = mid + 1; }
            else hi = mid - 1;
        }
        return ans == -1 ? "" : list.get(ans).value();
    }
}
```

Complexity

- Set O(1), get O(log n). Space O(total sets).

Pitfalls

- Missing key and timestamp before the first version both return "".